



**Université
Gustave Eiffel**

RAPPORT DE PROJET DE STATISTIQUE

Par Cephass AKIKMANE

Sommaire

1. Introduction
2. Nettoyage et organisation des données
3. Vérification de la nature des variables quantitatives
4. Étude de la relation entre les différentes variables
5. Identification des variables qui ont une influence sur le score user
6. Conclusion

INTRODUCTION

Le but de ce projet est de faire une analyse exploratoire et statistique d'un jeu de données que nous aurons au préalable choisi.

Dans ce projet, nous allons traiter la problématique suivante : ***“Quelles sont les fonctions qui ont tendance à influencer le score user ?”***

Pour ce faire, nous allons utiliser le data set suivant. [Video Games.csv](#) Il contient le nom, l'année, le genre, la plateforme, l'éditeur, les ventes par région et globale, la note des utilisateurs, le nombre d'utilisateurs qui ont noté les jeux, mais aussi la note donnée par les critiques et leur nombre. Dans ce jeu de données nous avons différents types de variables, certaines sont quantitatives comme les ventes globales, les ventes au Japon ou encore la variable cible **score user** et d'autres sont qualitatives comme la franchise, l'année bien que ça soit sous forme de chiffre, la plateforme, l'éditeur et d'autres variables que nous allons développer dans la suite de ce rendu.

Dans le cadre de cette étude, nous avons appliqué différentes méthodes statistiques et graphiques pour :

1. Explorer et nettoyer les données (gestion des valeurs manquantes, échantillonnage, etc.) ;
2. Vérifier la distribution des variables quantitatives et déterminer si elles suivent une loi normale (tests paramétriques ou non-paramétriques) ;
3. Étudier les relations entre variables (corrélations, comparaisons de moyennes, visualisations type boîte à moustaches) ;
4. Identifier les facteurs ou variables qui ont une influence sur la variable cible score user , en analysant notamment l'effet du genre, du publisher, de la plateforme et du niveau de ventes.

C'est à la suite de ces différentes étapes que nous allons tirer des conclusions sur la problématique qui a été posée.

2. Nettoyage et organisation des données

Tout d'abord, nous procédons au nettoyage des données, pour ce faire, nous allons d'abord importer notre base de données puis retirée les variables dont nous n'avons pas besoin ici ce sont les colonnes 11,12,15 et 16 qui représente respectivement les variables suivantes critic_score critic_count Developpeur et Rating.

Après cela, nous ferons en sorte que les valeurs vide et les valeurs avec le champ de caractère N/A soit considéré en tant que valeur manquante NA puis nous ferons un na omit pour les supprimer et vérifieront avec un view() si cela à bien était fait.

```
library(dplyr)
library(stringr)
library(questionr)#charge l'extension questionr

data <- read.csv("C:/Users/wacin/Documents/JEUX DE DONEE/Video_Games.csv", header = TRUE, sep = ",")

data2 <- data[, -c(11,12,15,16)]

data2[data2 == ""] <- NA

data2[data2 == "N/A"] <- NA |

data3 <- na.omit(data2)

View(data3)
```

Après cela, nous allons vérifier les valeurs uniques de la variable Name ensuite, nous allons créer une nouvelle Colonne appeler Franchise qui contiendra le nom des franchises principales, pour ce faire nous utiliseront la ligne de code suivante

```
str_detect(Name, regex("Mario", ignore_case = TRUE)) ~ "Mario",
```

Ici str_detect permet de renvoyer une valeur True False ou Na ce qui permet de vérifier les expressions régulières définit avec regex tout en ignorant les majuscules et minuscule avec la fonction ignore_case et pour finir, nous allons donner un nom à cette valeur ici sa sera Mario, nous allons répéter la manipulation pour toutes les autres franchises principale que nous avons pu repérer.

```

unique(data3$Name)
data4<-data3 %>% mutate(Franchise=case_when(
  str_detect(Name, regex("Mario", ignore_case = TRUE)) ~ "Mario",
  str_detect(Name, regex("Pokemon|Pokémon|Poké", ignore_case = TRUE)) ~ "Pokémon",
  str_detect(Name, regex("Zelda|The Legend of Zelda", ignore_case = TRUE)) ~ "The Legend of Zelda",
  str_detect(Name, regex("Animal Crossing", ignore_case = TRUE)) ~ "Animal Crossing",
  str_detect(Name, regex("Donkey Kong", ignore_case = TRUE)) ~ "Donkey Kong",
  str_detect(Name, regex("Kirby", ignore_case = TRUE)) ~ "Kirby",
  str_detect(Name, regex("Wii Sports|Wii Play|Wii Fit|Wii Party", ignore_case = TRUE)) ~ "Wii Series",
  str_detect(Name, regex("Grand Theft Auto", ignore_case = TRUE)) ~ "Grand Theft Auto",
  str_detect(Name, regex("Call of Duty", ignore_case = TRUE)) ~ "Call of Duty",
  str_detect(Name, regex("Final Fantasy", ignore_case = TRUE)) ~ "Final Fantasy",
  str_detect(Name, regex("Need for Speed", ignore_case = TRUE)) ~ "Need for Speed",
  str_detect(Name, regex("Resident Evil", ignore_case = TRUE)) ~ "Resident Evil",
  str_detect(Name, regex("The Sims", ignore_case = TRUE)) ~ "The Sims",
  str_detect(Name, regex("Assassin's Creed", ignore_case = TRUE)) ~ "Assassin's Creed",
  str_detect(Name, regex("Halo", ignore_case = TRUE)) ~ "Halo",
  str_detect(Name, regex("Metal Gear Solid", ignore_case = TRUE)) ~ "Metal Gear Solid",
  str_detect(Name, regex("Monster Hunter", ignore_case = TRUE)) ~ "Monster Hunter",
  str_detect(Name, regex("Tekken", ignore_case = TRUE)) ~ "Tekken",
  str_detect(Name, regex("Tomb Raider", ignore_case = TRUE)) ~ "Tomb Raider",
  str_detect(Name, regex("Just Dance", ignore_case = TRUE)) ~ "Just Dance",
  str_detect(Name, regex("Street Fighter", ignore_case = TRUE)) ~ "Street Fighter",
  str_detect(Name, regex("God of War", ignore_case = TRUE)) ~ "God of War",
  str_detect(Name, regex("Battlefield", ignore_case = TRUE)) ~ "Battlefield",
  str_detect(Name, regex("NBA 2K", ignore_case = TRUE)) ~ "NBA 2K",
  str_detect(Name, regex("Madden NFL", ignore_case = TRUE)) ~ "Madden NFL",
  str_detect(Name, regex("FIFA", ignore_case = TRUE)) ~ "FIFA",
  str_detect(Name, regex("Age of Empires", ignore_case = TRUE)) ~ "Age of Empires",
  str_detect(Name, regex("Lego", ignore_case = TRUE)) ~ "Lego ",
  str_detect(Name, regex("Crash Bandicoot", ignore_case = TRUE)) ~ "Crash Bandicoot",
  str_detect(Name, regex("Gran Turismo", ignore_case = TRUE)) ~ "Gran Turismo",
  str_detect(Name, regex("Uncharted", ignore_case = TRUE)) ~ "Uncharted",
  str_detect(Name, regex("Gears of War", ignore_case = TRUE)) ~ "Gears of War",
  str_detect(Name, regex("Fallout", ignore_case = TRUE)) ~ "Fallout",
  str_detect(Name, regex("Far Cry", ignore_case = TRUE)) ~ "Far cry ",
  str_detect(Name, regex("Batman", ignore_case = TRUE)) ~ "Batman",
  TRUE ~ "Autre Franchise/Non Classé"
))

```

Pour la Franchise “Wii series” vous pouvez observer plusieurs expressions régulière, cela et dû au fait que certains jeux ont dans leur nom Wii comme “PokéPark Wii: Pikachu's Adventure” par exemple.

Nous faisons apprêt la même chose pour les plateformes, nous vérifions d'abord les valeurs uniques de la variable platform puis, on crée une nouvelle colonne appeler console_creator à une différence près ici, nous allons simplement trier les console dans différente valeur par exemple PS3,PS4 dans Sony ds et 3 ds dans Nintendo après cela nous réorganisent notre data set dans l'ordre suivant pour une lecture plus simple : Name, Franchise,Platform,Console_creator,year of reales,genre,publisher,NA sales,Eu sales, Jp sales , other sales , global sales,user score et user count

```

unique(data3$Platform)
data5<-data4%>% mutate(Console_creator=case_when(
  str_detect(Platform,regex("psv|ps3|ps4|ps|psp|ps",ignore_case =TRUE))~ "Sony",
  str_detect(Platform,regex("wii|ds|3ds|wiiu|gc|gba",ignore_case = TRUE))~"Nintendo",
  str_detect(Platform,regex("xone|x360|xb",ignore_case = TRUE))~"Microsoft",
  TRUE~"Autre Createur/Non Classé"
))

data6<-data5[,c(1,13,2,14,3,4,5,6,7,8,9,10,11,12)];

```

3. Vérification de la nature des variables quantitatives

Après avoir nettoyé et organisé les données, nous procédons à une vérification de la nature des variables quantitative, pour ce faire, nous commençons par convertir la variable User_Score qui était une chaîne de caractère en variable numérique, pour ce faire, nous commençons par effacer les valeurs non numérique comme "tbd" en les remplaçant par NA.

```
data6$User_Score[data6$User_Score == "tbd"] <- NA
```

Après avoir remplacé les valeurs non numériques par NA, nous pouvons procéder à la conversion en numérique de la variable pour pouvoir effectuer les calculs statistiques.

```
data6$User_Score<- as.numeric(data6$User_Score)
```

Nous procédons ensuite au test de normalité des variables quantitatives en utilisant le test de Shapiro. Pour ce faire, nous prenons un échantillon de 5 000 individus, car c'est la limite possible pour effectuer ce test

```
shapiro.test(data6_sample$User_Score)
```

Nous procédons ainsi pour toutes les variables quantitatives.

```
> shapiro.test(data6_sample$User_Score)
```

```
Shapiro-Wilk normality test
```

```
data: data6_sample$User_Score  
W = 0.29435, p-value < 2.2e-16
```

```
> shapiro.test(data6_sample$User_Count)
```

```
Shapiro-Wilk normality test
```

```
data: data6_sample$User_Count  
W = 0.25479, p-value < 2.2e-16
```

```
> shapiro.test(data6_sample$Global_Sales)
```

```
Shapiro-Wilk normality test
```

```
data: data6_sample$Global_Sales  
W = 0.29435, p-value < 2.2e-16
```

```
> |
```

Les tests de normalité indiquent que toutes les variables analysées rejettent l'hypothèse de normalité, comme le montre la p-valeur inférieure à 0,05 dans les résultats.

4. Étude de la relation entre les différentes variables

Tout d'abord, nous procédons d'abord à une corrélation entre les variables quantitatives pour pouvoir évaluer la dépendance entre ces variables.

```
num_vars <- data6 %>%  
  select(User_Score, User_Count, Global_Sales)  
cor(num_vars, use="complete.obs", method="spearman")
```

```
      User_Score User_Count Global_Sales  
User_Score  1.0000000  0.1789015  0.1490874  
User_Count  0.1789015  1.0000000  0.3944672  
Global_Sales 0.1490874  0.3944672  1.0000000  
> |
```

Les résultats suivants nous montrent qu'il y a une faible corrélation entre les variables : entre User_Score et User_Count, la corrélation est inférieure à 0,3, ce qui veut dire que le nombre d'avis ne dépend pas fortement de la note attribuée par les joueurs ; entre User_Score et Global_Sales, la corrélation est également inférieure à 0,3, ce qui veut aussi dire que les ventes globales ne sont pas directement liées aux notes des utilisateurs ; enfin, entre User_Count et Global_Sales, la corrélation reste inférieure à 0,5, indiquant que les jeux ayant plus d'utilisateurs ont tendance à avoir de meilleures ventes, sans que le lien soit très fort.

Après avoir fait cela Pour mieux comprendre comment les notes des utilisateurs varient selon certaines caractéristiques des jeux, nous avons calculé les moyennes de User_Score pour nos différentes variables qualitatives, pour ce faire, nous allons procéder de la manière suivante d'abord, nous créons un nouveau tableau pour y stocker nos resulta apres cela nous

regroupons les valeurs par variable à l'aide de la fonction `group_by()` puis nous procédons au calcul de la moyenne en utilisant la fonction `summarise` de cette manière :

```
franchise_scores <- data6 %>%
  group_by(Franchise) %>%
  summarise(mean_score = mean(User_Score, na.rm = TRUE),
            n = n())
```

Nous créons une nouvelle variable appelée `mean_score`, qui contiendra la moyenne des scores calculée à partir de la fonction `mean()`. Ensuite, nous créons la variable `n`, qui nous permettra, grâce à la fonction `n()`, de savoir combien d'individus sont présents dans chaque groupe. Nous répétons ensuite ce processus pour toutes les variables qualitatives.

Nous créons ensuite une carte de chaleur pour **visualiser** les résultats de notre calcul de corrélation fait précédemment

```
corrplot(cor(num_vars, use="complete.obs", method="spearman"), method="color")
```

Ce qui donnera ce résultat

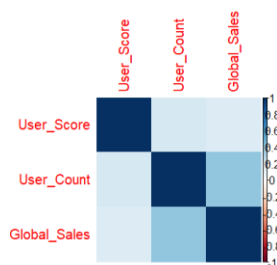
Nous remarquons que les

valeurs affichées sur la

carte de chaleur

correspondent à notre

calcul des corrélations.

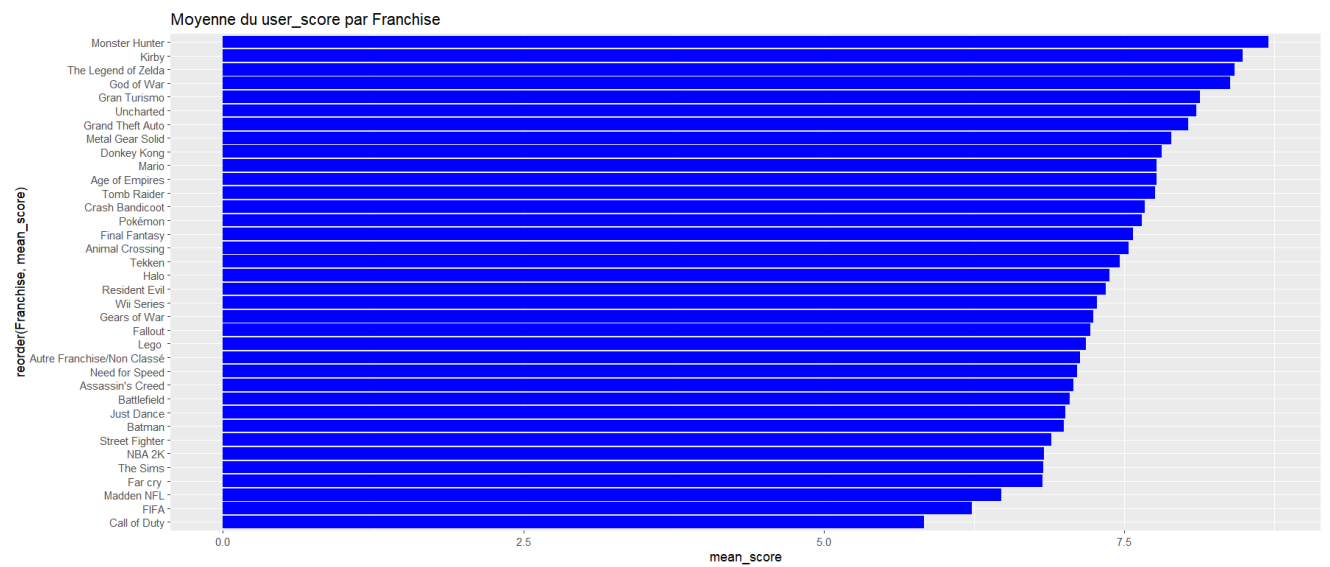


Nous créons ensuite des diagrammes en barres pour représenter les différentes moyennes calculées précédemment de la manière suivante

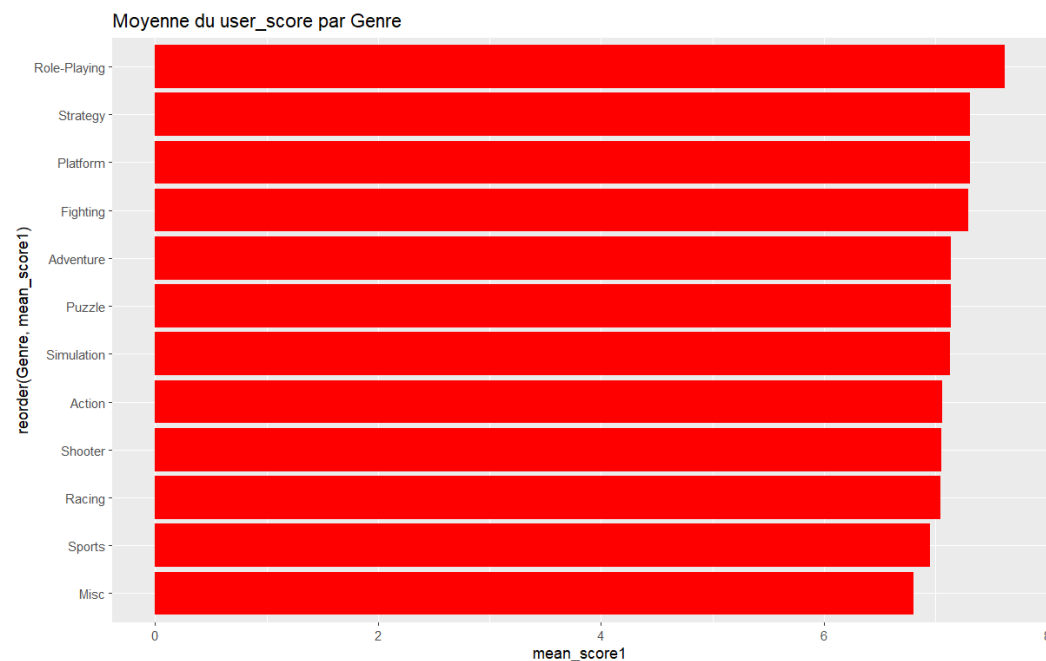
```
ggplot(franchise_scores, aes(x = reorder(Franchise, mean_score), y = mean_score)) +
  geom_col(fill="blue") +
  coord_flip() +
  labs(title="Moyenne du user_score par Franchise")
```

Dans un premier temps, nous utilisons `ggplot()` pour indiquer le tableau de données et les variables à représenter sur les axes X et Y. Ensuite, nous utilisons la fonction `geom_col()`, qui permet de créer un diagramme en barres où la hauteur de chaque barre correspond à la moyenne. Ensuite, la fonction `reorder()` permet de réorganiser les barres de la moyenne la plus basse à la plus élevée. Après, la fonction `coord_flip()` permet d'inverser les axes pour rendre les labels plus lisibles, et pour finir, la fonction `labs()` permet d'ajouter un titre au graphique. Nous appliquons ensuite la même procédure aux autres variables qualitatives, ce qui nous donne les résultats suivants :

Moyenne de user_score par Franchise



Moyenne de user_score par genre



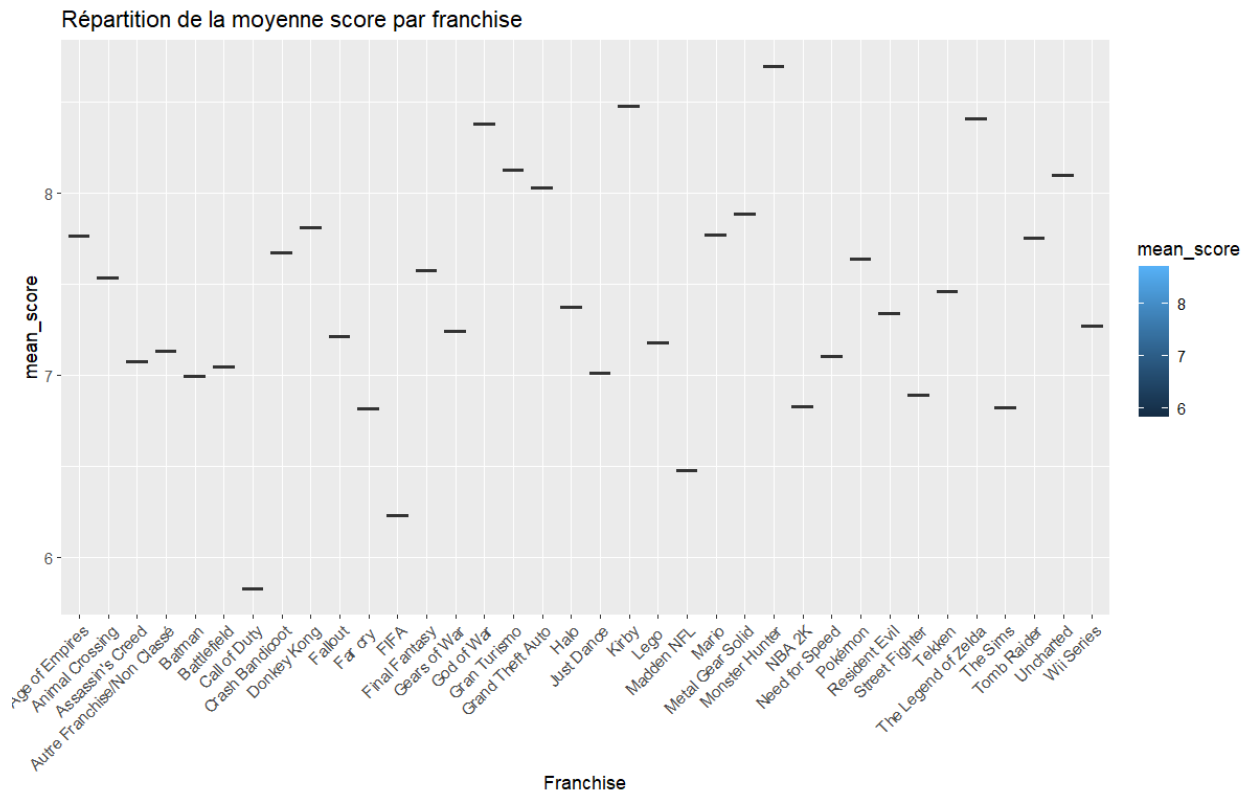
Nous réalisons ensuite des boxplots pour visualiser plus en détail la répartition des User_Score selon les groupes, car les graphiques précédents montraient uniquement les moyennes et ne permettaient pas d'observer la dispersion ni les valeurs extrêmes. Les boxplots permettent ainsi

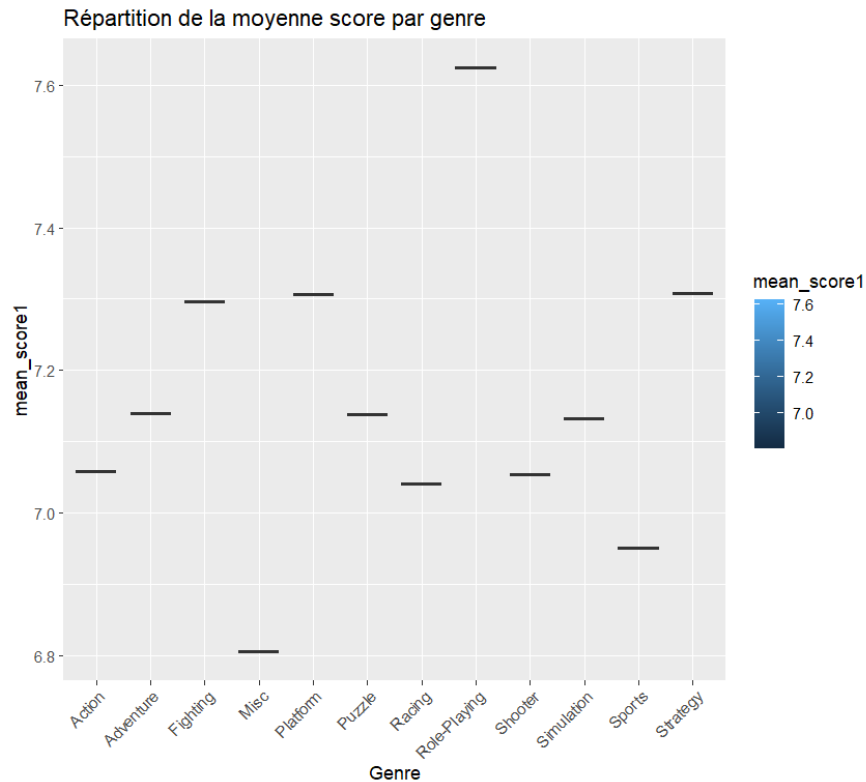
de mieux comprendre la répartition des scores par Franchise ou par Genre et d'identifier les valeurs particulières. Nous allons, pour ce faire, procéder à la création des boxplots pour visualiser plus en détail la répartition des User_Score par Franchise. Les étapes sont similaires à celles des graphiques précédents, mais ici, nous utilisons `geom_boxplot()` pour représenter les valeurs individuelles sous forme de boîtes, et `theme(axis.text.x = element_text(angle=45, hjust=1))` pour incliner les labels et améliorer leur lisibilité.

```
ggplot(franchise_scores, aes(x=Franchise, y=mean_score, fill=mean_score)) +  
  geom_boxplot() +  
  theme(axis.text.x = element_text(angle=45, hjust=1)) +  
  labs(title="Répartition de la moyenne score par franchise")
```

Nous referons la même manipulation pour la répartition de la moyenne score par genre

Ce qui donne les résultats suivants :





L'analyse des boxplots montre que la répartition des moyennes de User_Score ne suit pas une loi normale. On observe que certaines franchises ou genres présentent des scores particulièrement élevés, tandis que d'autres ont des scores plus bas, ce qui indique un écart important entre les groupes. De plus, la répartition des moyennes selon le type de jeu n'est pas constante et présente des variations marquées, avec certaines valeurs particulières par rapport à l'évolution générale. Ces résultats confirment que les scores des utilisateurs varient fortement selon la franchise ou le genre du jeu.

Pour finir, nous allons vérifier si les différences entre groupes sont significatives ou non. Pour ce faire, nous procédons à des tests de Kruskal-Wallis sur User_Score selon les variables Franchise et Platform. Nous avons ensuite appliqué le test de Dunn avec correction de Bonferroni pour comparer chaque groupe à tous les autres groupes.

```
kruskal.test(User_Score~Franchise, data=data6_sample)
res <- dunnTest(User_Score ~ Franchise, data = data6_sample, method = "bonferroni")
```

Après cela nous transformons les résultats en data frame

```
df_res <- as.data.frame(res$res)
```

```
sig <- df_res %>% filter(P.adj < 0.05) %>%  
  arrange(P.adj)  
  
print(sig)
```

User_Score par Franchise (ordonné par médiane)

Franchise	Min	Q1	Médiane	Q3	Max	Outliers
of Empires	0	0	0	0	0	
omb Raider	0	0	0	1	1	
ion Classé	0	0	0	5	5	10, 15, 18, 20
resident Evil	0	0	2	3	3	4
Batman	0	0	2	3	3	
Gear Solid	0	0	2	3	3	4
Just Dance	0	0	2	3	3	4
FIFA	0	0	2	3	3	4
The Sims	0	0	2	3	3	4
Lego	0	0	2	3	3	4
ial Fantasy	0	0	2	3	3	4
for Speed	0	0	2	3	3	4
Tekken	0	0	2	3	3	4
NBA 2K	0	0	2	3	3	4
reet Fighter	0	0	2	3	3	4
in's Creed	0	0	2	3	3	4
adden NFL	0	0	2	3	3	4
Bar cry	0	0	2	3	3	4
nkay Kong	0	0	2	3	3	4
Theft Auto	0	0	2	3	3	4
Pokémon	0	0	2	3	3	4
Kirby	0	0	2	3	3	4
Battlefield	0	0	2	3	3	4
Bandicoot	0	0	2	3	3	4
ars of War	0	0	2	3	3	4
Mario	0	0	2	3	3	4
Wii Series	0	0	19	20	20	82
Fallout	0	0	2	3	3	4
nd of Zelda	0	0	2	3	3	4
ater Hunter	0	0	2	3	3	4
Call of Duty	0	0	2	3	3	4
al Crossing	0	0	2	3	3	4
God of War	0	0	2	3	3	4
Uncharted	0	0	2	3	3	4
Halo	0	0	2	3	3	4
an Turismo	0	0	2	3	3	4

12

5. Identification des variables qui ont une influence sur le score user

D'après les analyses que nous avons fait sur l'échantillon data6_sample et la base complète data6, plusieurs relations statistiquement significatives émergent :

- Relation entre les variables quantitatives : la matrice de corrélation de Spearman montre des corrélations faibles à modérées entre User_Score, User_Count et Global_Sales (coefficients ≈ 0.15 à 0.39), indiquant des associations monotones mais pas de forte colinéarité entre ces variables numériques.
- Dunn avec la correction Bonferroni : les tests post-hoc montrent quelles paires de groupes diffèrent concrètement. Exemples notables :
Pour Genre, beaucoup de paires impliquant Adventure ou Strategy présentent des différences significatives (après correction Bonferroni).
Pour Franchise, des comparaisons impliquant « Autre Franchise/Non Classé » et certaines franchises (ex. Mario, Call of Duty, FIFA, Wii Series) sont significatives.
Pour Console_creator, Sony se distingue significativement de Nintendo et Microsoft (comparaisons avec p-ajustées très faibles).

```
> kruskal.test(User_Score~Genre, data=data6_sample)

Kruskal-Wallis rank sum test

data: User_Score by Genre
Kruskal-Wallis chi-squared = 211.05, df = 11, p-value < 2.2e-16

> kruskal.test(User_Score~Console_creator, data=data6_sample)

Kruskal-Wallis rank sum test

data: User_Score by Console_creator
Kruskal-Wallis chi-squared = 404.45, df = 3, p-value < 2.2e-16

> kruskal.test(User_Score~Publisher, data=data6_sample)

Kruskal-Wallis rank sum test

data: User_Score by Publisher
Kruskal-Wallis chi-squared = 1264.7, df = 236, p-value < 2.2e-16

> kruskal.test(User_Score~Franchise, data=data6_sample)

Kruskal-Wallis rank sum test

data: User_Score by Franchise
Kruskal-Wallis chi-squared = 522.06, df = 35, p-value < 2.2e-16
```

- Comparaisons par catégorie (Kruskal–Wallis) : les tests de Kruskal–Wallis indiquent des différences significatives du User_Score selon Genre, Console_creator, Franchise,

Publisher et Sales_Group (p-value < 2.2e-16 dans tous les cas). Ces résultats signifient que la distribution des scores utilisateurs varie entre au moins deux groupes pour chacune de ces variables.

On a fait un filtre de sur deux modalités en particulier afin d'avoir un échantillon de ce qui peut se passer sur une plus grosse base de données.

```
# 5) Mesure d'effet (Cliff's delta) pour une paire d'exemple :  
# remplacer "PC" et "PS4" par les plateformes qui t'intéressent  
groupA <- data6 %>% filter(Platform == "PC") %>% pull(User_Score)  
groupB <- data6 %>% filter(Platform == "PS4") %>% pull(User_Score)  
cliff <- cliff.delta(groupA, groupB)  
print(cliff)
```

Cliff's Delta

```
delta estimate: 0.1698317 (small)  
95 percent confidence interval:  
      lower      upper  
0.09375977 0.24393210
```

On a par exemple pc contre ps4 donne un **Cliff's Delta = 0.1698** (IC95% $\approx$ [0.094, 0.244]) qui est un effet positif faible : les scores PC sont légèrement supérieurs à ceux de PS4 en termes de rangs, mais l'effet reste petit au plan pratique.

- Ventes et score :Sales_Group montre des différences très significatives du User_Score entre quarts de ventes (Kruskal $p < 2.2e-16$), ce qui suggère une relation robuste entre popularité/ventes et note utilisateur ; à approfondir via corrélation partielle ou modèle pour contrôler d'autres facteurs.

Les corrections multiples (Bonferroni) sont strictes mais malgré ça on trouve des différences significatives ce qui renforce encore la crédibilité statistique. Publisher a beaucoup de modalités, les comparaisons multiples sur autant de publishers sont bruyantes ; regrouper les petits éditeurs en "Other" améliore l'interprétabilité. Les tests montrent *des différences de distribution*, pas nécessairement des relations causales. Pour identifier l'influence "indépendante" d'une variable il faut modéliser (régression, modèles réguliers, arbres) et vérifier multicollinéarité / diagnostics. Les tailles d'effet (Cliff's Delta, différences de médianes, eta/epsilon-squared pour KW) sont à rapporter en plus des p-values pour juger de l'importance pratique

6. Conclusion

Finalement, notre analyse nous a permis de mieux comprendre quels facteurs influencent réellement le User Score dans le domaine du jeu vidéo. Après avoir nettoyé et structuré le dataset, vérifié la nature des variables et appliqué des méthodes statistiques adaptées (Shapiro, corrélation de Spearman, Kruskal-Wallis, test de Dunn), on remarque que plusieurs éléments jouent un rôle significatif sur la note des utilisateurs. Même si les variables quantitatives comme les ventes ou le nombre d'avis montrent seulement une corrélation faible à modérée avec la note des joueurs, les variables qualitatives, elles, ressortent clairement. La franchise, la plateforme, le genre ainsi que le publisher ont un impact notable, confirmé par les tests statistiques. Certaines franchises célèbres ou plateformes comme celles de Sony se démarquent, tout comme certains genres de jeux. Au passage, les ventes ne semblent pas faire automatiquement un bon score, ce qui montre que popularité et qualité perçue ne vont pas forcément de pair. Cette étude nous montre que le choix du type de jeu, de la plateforme et de l'univers auquel il appartient influence plus la satisfaction des joueurs que son succès commercial. Le projet ouvre aussi la porte à des analyses plus poussées, notamment via des modèles de régression pour mesurer l'impact isolé de chaque variable et explorer une notion de causalité.

Ce Projet nous a permis de combiner data cleaning, visualisation, tests statistiques et interprétation critique, ce qui renforce clairement notre compréhension du comportement des joueurs dans l'industrie du gaming.